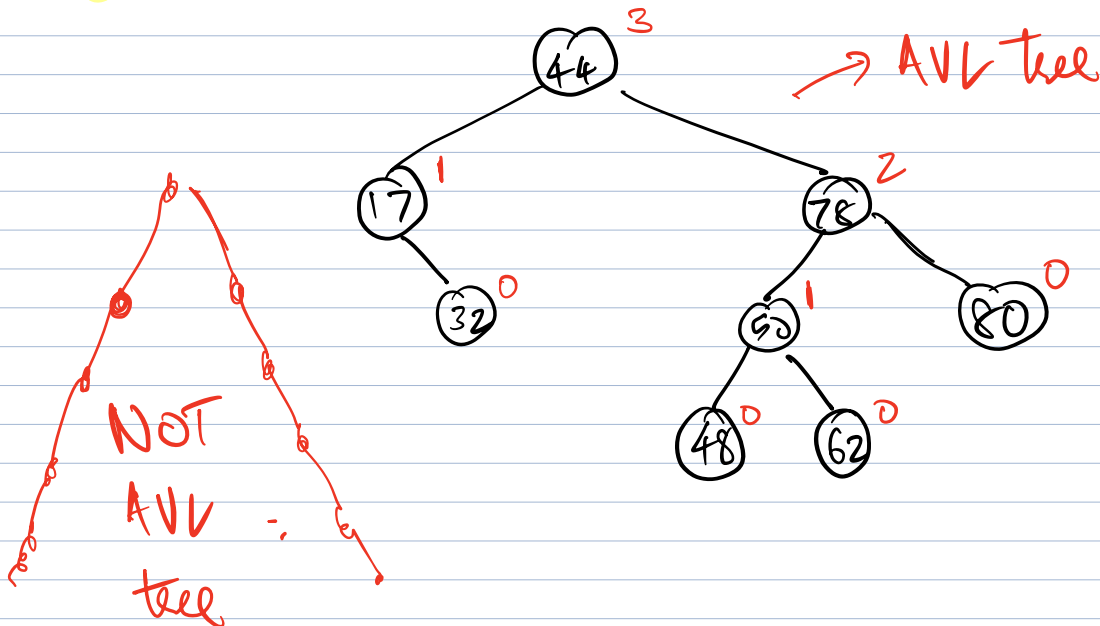


AVL Trees

Discovered in 1962 by Adelson-Velskii & Landis

AVL Trees are balanced binary search trees. They satisfy all the properties of BST's. In addition, they satisfy some additional structural properties.

Additional property: For every node v of the tree, the heights of subtrees of v can differ by at most 1.



Claim: The height of an AVL tree with n nodes is $O(\log n)$.

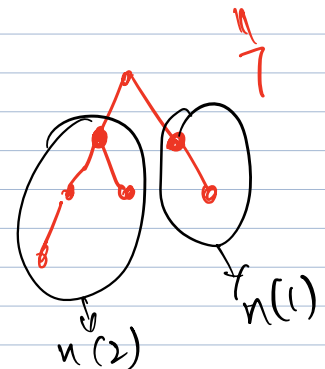
Proof 1: let $n(h)$ be the smallest no. of nodes that can be present in an AVL tree of height h .

$$n(1) = 2$$

$$n(2) = 4$$

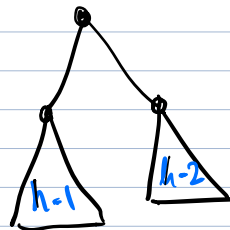


What's $n(3)$?



For $h \geq 3$, we have a root plus one subtree of height $h-1$ and one subtree of height $h-2$ (AT least)

$$\text{So } n(h) = 1 + n(h-1) + n(h-2)$$



Since $n(h-1) \geq n(h-2)$, we have.

$$n(h) \geq 2n(h-2).$$

$$n(h) > 2 n(h-2)$$

$$> 2 \cdot 2 \cdot n(h-2-2) = 4 n(h-4)$$

$$> 8 n(h-6)$$

⋮

$$> 2^i n(h-2^i)$$

$$\text{Set } i = \frac{h-1}{2} \Rightarrow n(h) > 2^{\frac{h-1}{2}} n(1)$$

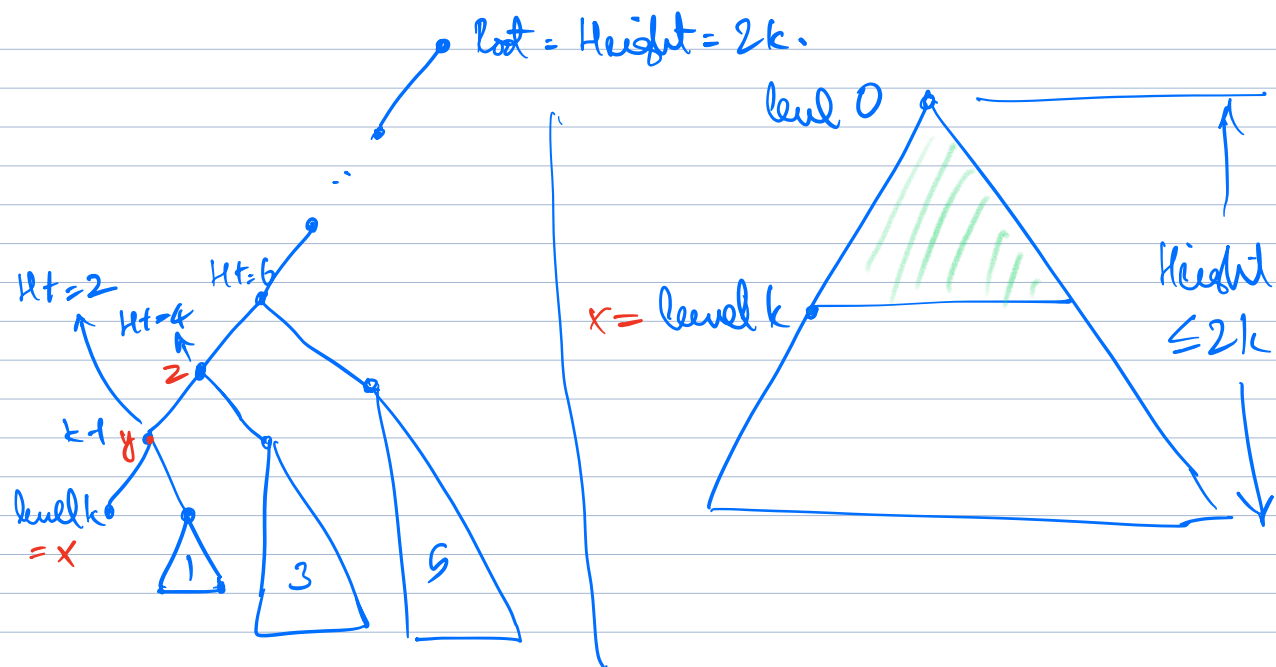
$$> 2^{h/2} = (2^{1/2})^h \\ = (\sqrt{2})^h$$

$$\text{So } n(h) > 2^{h/2} \Rightarrow \boxed{h < 2 \log n}$$

So height is $O(\log n)$.

Another way to see this.

Consider the leaf of the least depth. Let x be a leaf at level/depth k . Then we will see that all the leaves are at level at most $2k$.



The tree is completely full upto depth k .

$$2^{k+1} - 1 \leq n \leq 2^{2k+1} - 1$$

Proof 2: We saw $n(h) = n(h-1) + n(h-2) + 1$

We will try to find the largest c for which

$$n(h) \geq c^h.$$

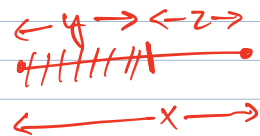
That is, largest c for which $c^{h-1} + c^{h-2} \geq c^h$

We want c such that $c^h \leq c^{h-1} + c^{h-2}$

That is $c^2 - c - 1 \leq 0$

Solving quadratic equation, we get two roots for $c^2 - c - 1 = 0 \Rightarrow (1 \pm \sqrt{5})/2$.

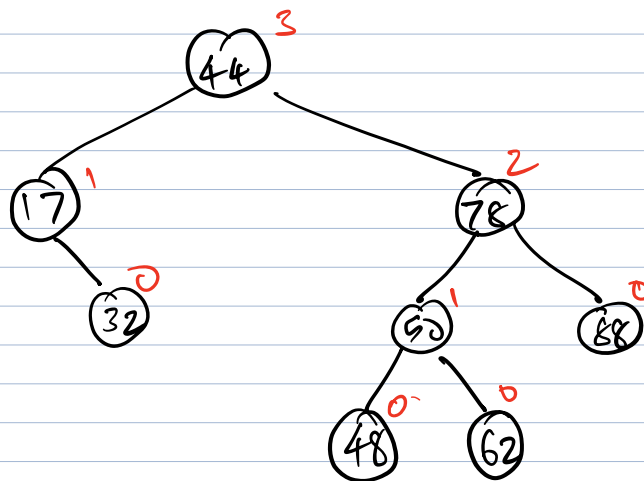
$\frac{1+\sqrt{5}}{2} \approx 1.618 \rightarrow \text{Golden Ratio}$



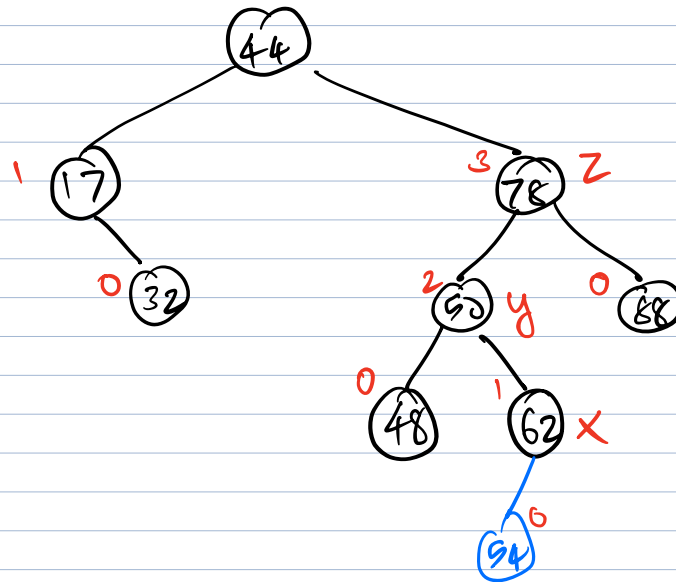
So $n(h) \geq 1.618^h$

And $h \leq \log_{1.618} n \approx \underline{1.44 \log n}$
 $\rightarrow O(\log n)$

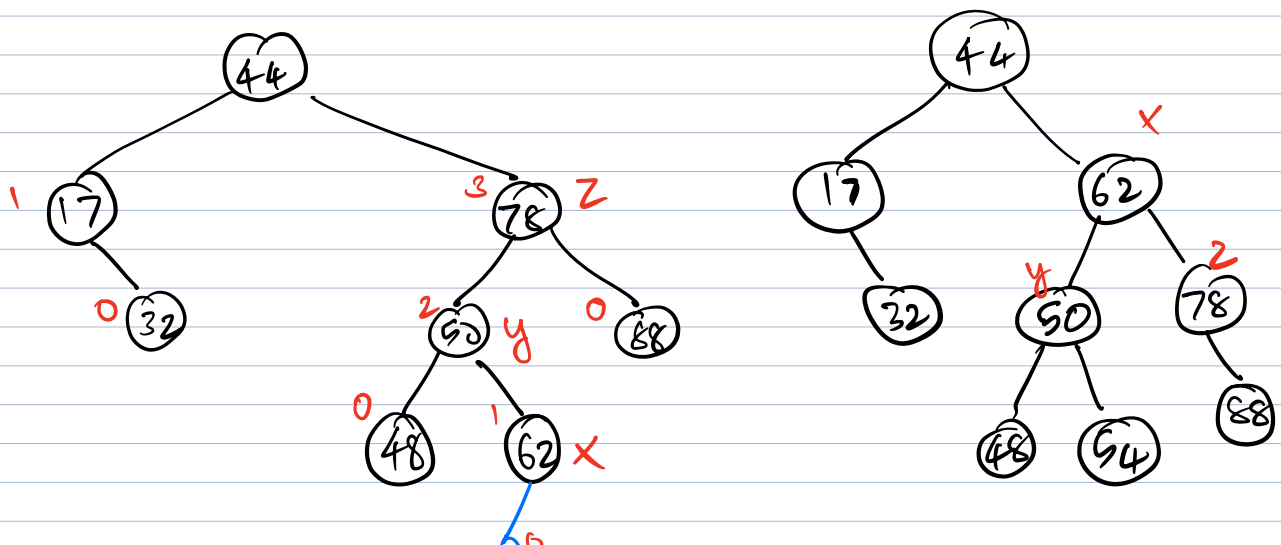
How do we maintain the structure?



Suppose we need to insert 94 to the above tree



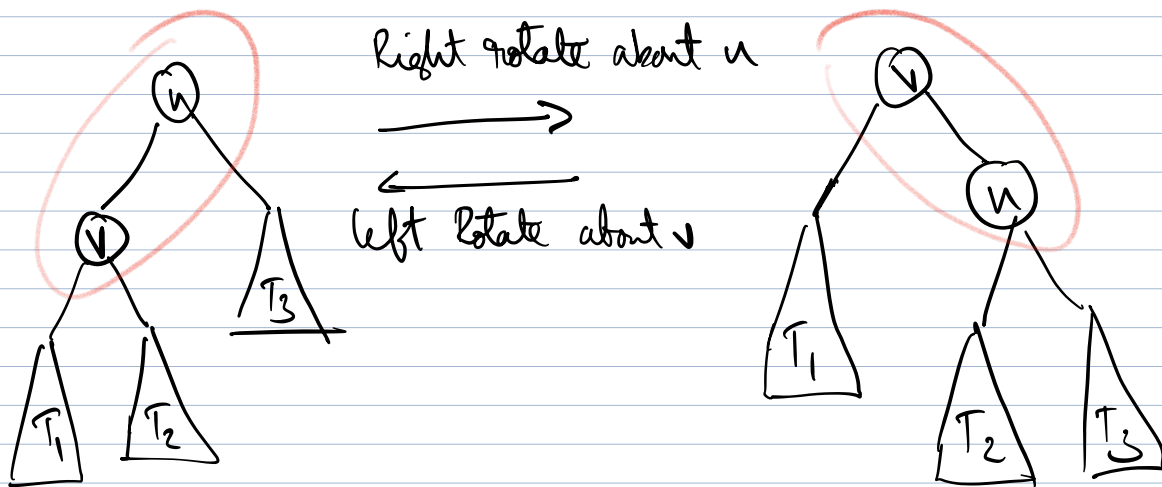
Height balance issue may arise in an ancestor of the inserted node. Here, in the above tree, the unbalance occurs in the node with key 78.



(94)

Rotation

This is a way to reorganize a BST.

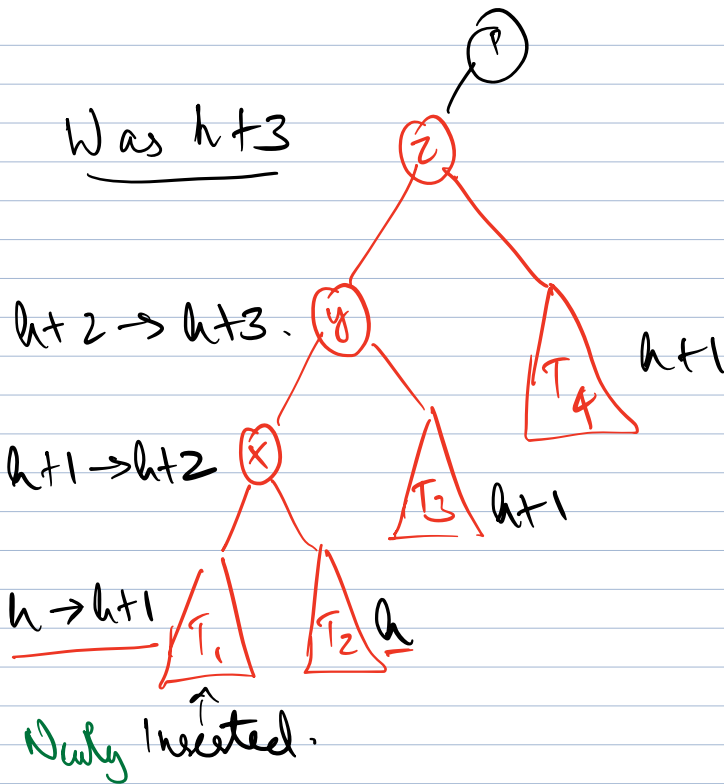


Note: This satisfies BST properties -

We need to update a constant (6) pointers.

Situation: We have inserted a new value into the AVL tree. It has resulted in an imbalance at node z . Let y, x be the child and grandchild of z (respectively) along the path to the newly inserted node.

Was $h+3$



This is the case when y is left child of z and x is left child of y .



Suppose height of x , y , z went up.

$h(T_2) \neq h+2$ because it was originally balanced.

$h(T_2) \neq h+1$ because height of x does not increase if $h(T_2) = h+1$.

So $h(T_2) = h$.

So $ht(x) = h+1$ to $h+2$.

Since y is balanced now, $h(T_3) = h+1, h+2$ or $h+3$.

Since y was balanced before, $h(T_3) \neq h+3$.

Since $ht(y)$ changes, $h(T_3) \neq h+2$.

So $h(T_3) = h+1$.

So $h(y) = h+2 \rightarrow h+3$.

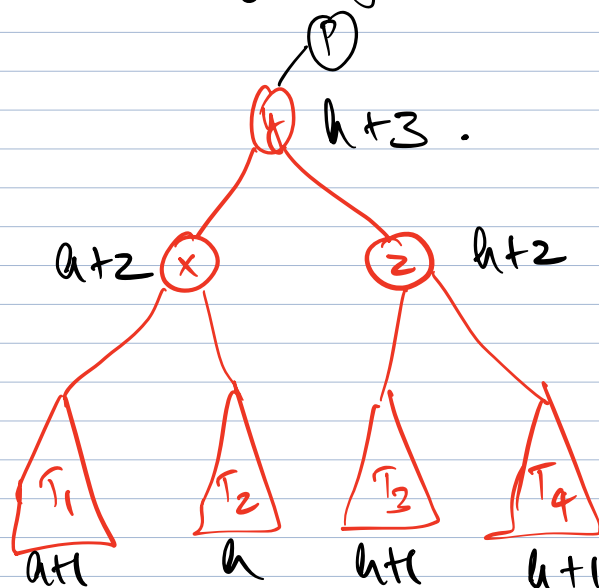
z just became imbalanced.

So $h(T_4)$ must be $h+1$.

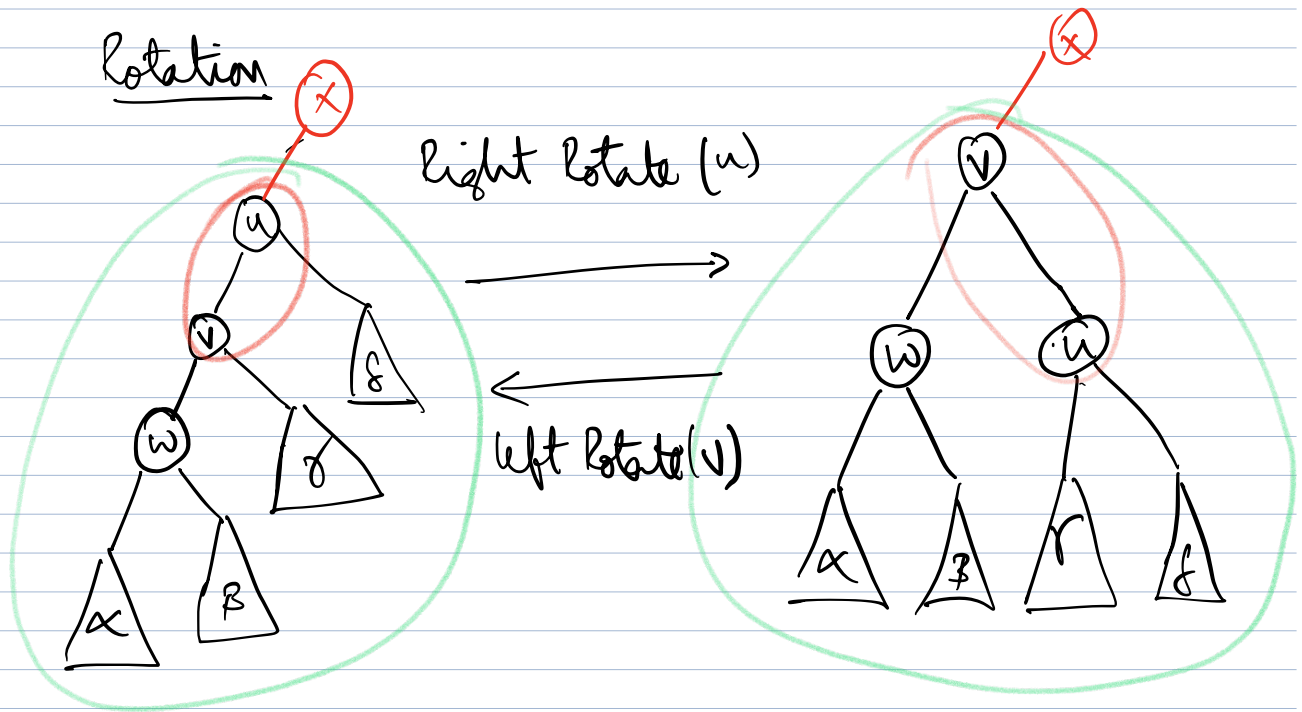
So $h(z)$ was originally $h+3$.

Time taken:

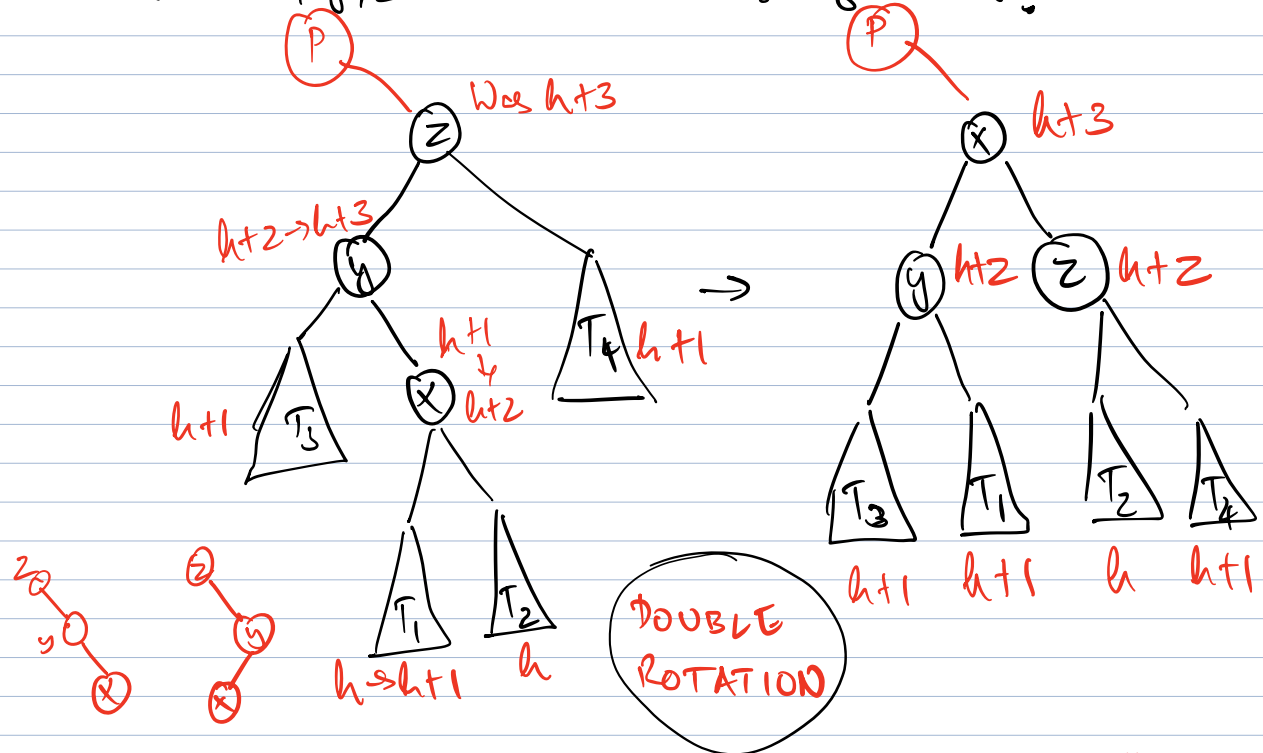
1 rotation, but
we may need
to spend $O(h)$
 $= O(\log n)$ time
to search for the
imbalance



Rotation



What if x, y, z were not in a straight line?

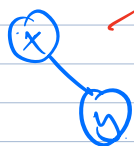


Deletion

Once you do single or double rotation, there are no more imbalances left.

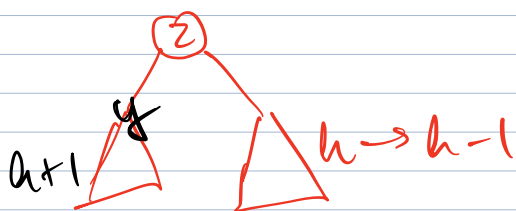
The ^{actual} deleted node will be either a leaf node or a parent of a leaf node. Why?

Either deleted node is a leaf, or it has only one child.

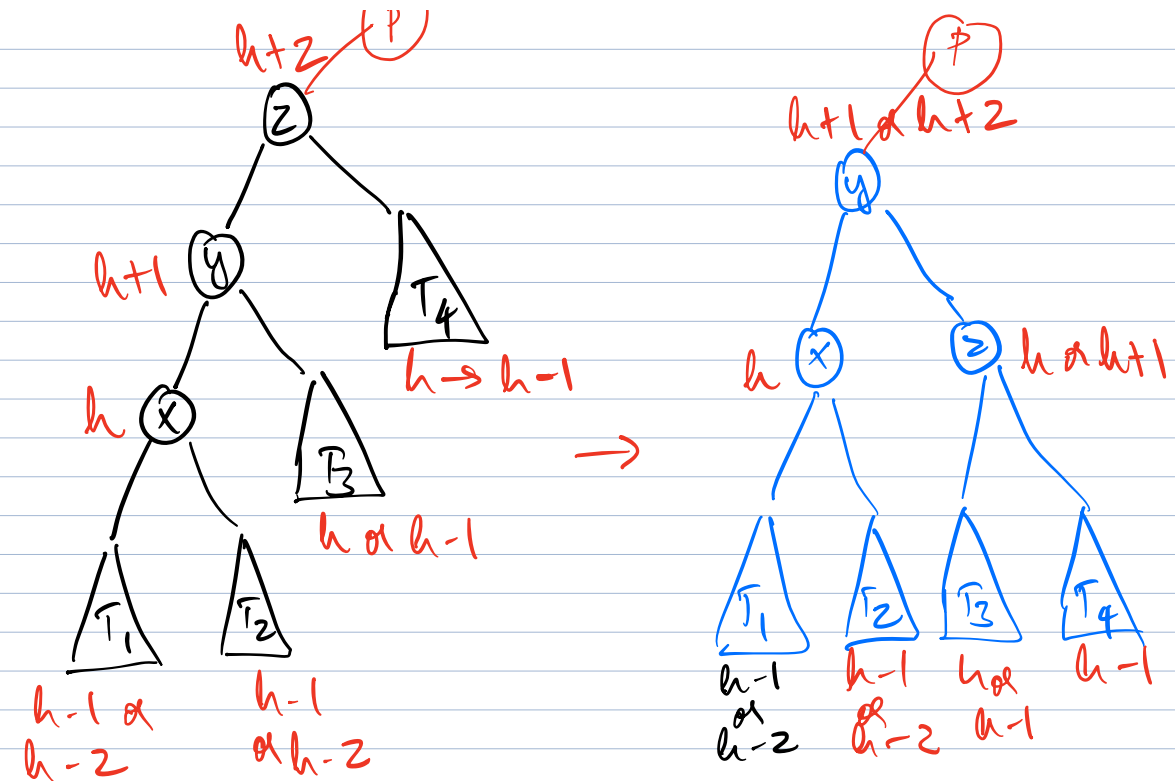


If x has only one child y , then AVL properties imply that y has no children.

Let z be the first unbalanced ancestor of the deleted node. Let y be the child of z with larger height and let x be the child of y with larger height (choose x to be in the same line as $y-z$ if heights are the same).



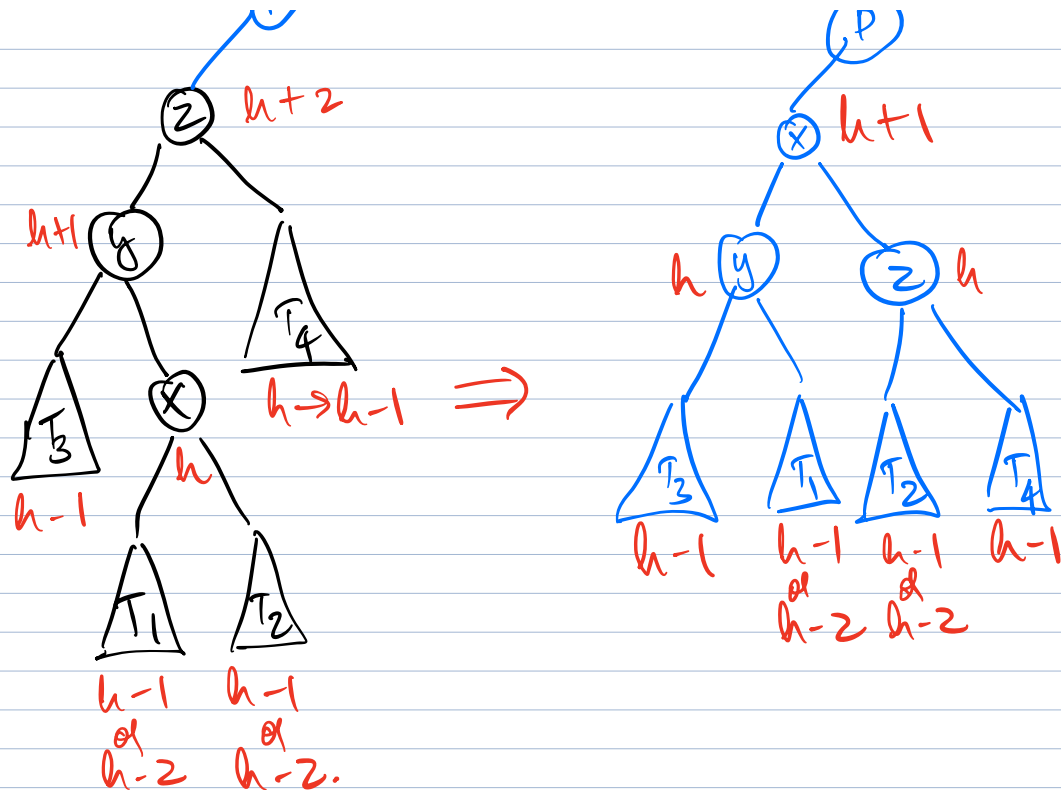
Say one subtree of z was at height h and it got reduced to $h-1$. The height of the other subtree must be $h+1$.



One of T_1 or T_2 has to be height $h-1$.

We may need to repeat this going upwards.

Because height of this entire tree may fall from $h+2$ to $h+1$. If the sibling tree had height $h+3$, it creates imbalance at P . If the sibling tree had height $h+1$, then the height of P goes down and we may need to check the parent of P .



$$h(T_3) = h-1 \text{ or } h.$$

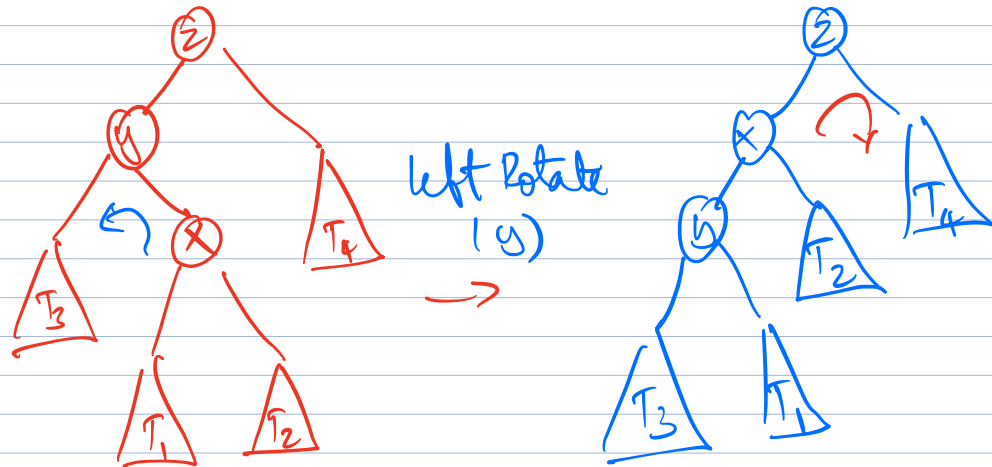
If $h(T_3) = h$, then we would have picked root of T_3 as x . So we may assume $h(T_3) = h-1$.

Since the height of the entire tree goes down, we will need to check for imbalance at p , the original parent of z . If there is imbalance, we need to do the same process at the original parent of z . If no imbalance, we may need to check if height of p changes

.

Double Rotation

and if so, there is imbalance at parent of p.



Right
Rotate(z)

